AI Academy, National AI Center
AI-ENG-201 – Machine Learning – Fall 2026

Homework 1 – k-Nearest Neighbors Report

Name: Fatima Alibabayeva **Student ID:** Not provided
Submitted: 5 June **Late days used:** 0
Total time: ≈14 hours **Collaborators:** None

I, Fatima Alibabayeva, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in the assignment. I can explain every line of code and every paragraph of writing in my submission.

Signed: Fatima Alibabayeva **Date:** 5 June

AI tool usage declaration: Used AI assistance only for editing, consistency checks, and formatting after drafting; all code behavior and reported metrics were verified by running the submitted scripts.

Executive summary

The best single-split validation model is KNN with $k = 21$, selected only by validation F1. On validation it obtains accuracy 0.7433, precision 0.7463, recall 0.5000, F1 0.5988, and AUC-ROC 0.7169; on the held-out test set used once at the end it obtains accuracy 0.6730, F1 0.5222, and AUC-ROC 0.7349. The model clearly beats the majority baseline, whose F1 is 0.0000 and AUC is 0.5000. Cross-validation on the train+validation data prefers $k = 5$ with mean CV F1 0.5849, showing that the single validation split has some selection variance. Standardizing features inside each training fold improves best mean CV F1 from 0.5849 to 0.7196 because unscaled fare values dominate Euclidean distance.

1 Exploratory Data Analysis [20 pts]

1.1 Summary statistics [4 pts]

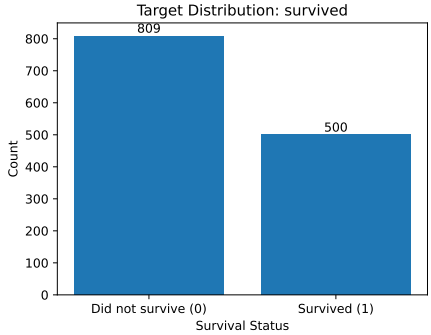
The Vanderbilt Titanic dataset has $N = 1309$ passengers. Numeric summaries are computed before imputation.

Feature	Mean	Median	Std	Non-null	Missing
pclass	2.295	3.000	0.838	1309	0
age	29.881	28.000	14.413	1046	263
sibsp	0.499	0.000	1.042	1309	0
parch	0.385	0.000	0.866	1309	0
fare	33.295	14.454	51.759	1308	1

Feature	Value	Count	%
sex	male	843	64.4
sex	female	466	35.6
embarked	S	914	69.8
embarked	C	270	20.6
embarked	Q	123	9.4
embarked	missing	2	0.2

Age has 20.1% missingness and fare has one missing value. Fare is strongly right-skewed: its mean is 33.30 but its median is only 14.45.

1.2 Target distribution and class balance [4 pts]



Class proportions are survived = 0.38 and did not survive = 0.62. I consider the dataset **mildly imbalanced**: the minority class is large enough for standard metrics to be meaningful, but the 62/38 split makes F1 more informative than accuracy because a majority predictor already reaches 61.69% validation accuracy.

1.3 Three feature-vs-target investigations [4 pts]

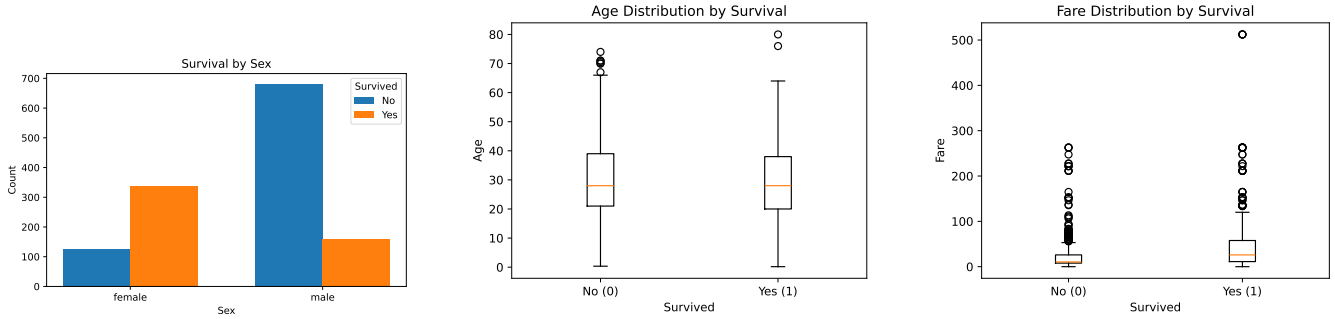


Figure 1: Feature-vs-target plots for sex, age, and fare.

Sex is highly predictive: female survivors substantially outnumber female non-survivors, while the reverse is true for males. Age alone is weaker because both survival groups have median age about 28. Fare is more informative: survivors have median fare about 26.0, while non-survivors have median fare about 10.5, reflecting the link between class, fare, and survival.

1.4 Missing-data strategy [4 pts]

For honest evaluation, imputation values in the experiment script are fit on the training partition only and then applied to validation/test. In cross-validation, imputation is fit inside each training fold. Age and fare use median imputation because both are numeric and fare is skewed; embarked uses mode imputation because it is categorical and has only 2 missing values. Cabin is dropped because 77.5% of its values are missing, above the 30% threshold.

1.5 Encoding and final feature matrix [4 pts]

Sex is binary encoded as male $\mapsto 1$, female $\mapsto 0$. Embarked is one-hot encoded as `embarked_C`, `embarked_Q`, and `embarked_S` because ports have no natural order. Pclass is kept ordinal because first, second, and third class have a meaningful ranking. The final matrix has $N = 1309$ and $p = 9$ features: pclass, sex, age, sibsp, parch, fare, and three embarked indicators.

2 KNN from Scratch [30 pts]

2.1 Class API [8 pts]

`src/knn.py` implements `KNN(k=5, metric="euclidean", q=2.0, task="classification", weights="uniform")` with `fit`, `predict`, and `predict_proba`. Public methods include type hints and docstrings. `fit` memorizes X and y; `predict` supports classification and regression; `predict_proba` returns an $(n_{queries}, n_{classes})$ probability matrix for classification.

2.2 Vectorized distance computation [8 pts]

Distances are computed by broadcasting X_{query} from (n_q, p) to $(n_q, 1, p)$ and X_{train} from (N, p) to $(1, N, p)$. The subtraction creates a full (n_q, N, p) difference tensor, then Euclidean, Manhattan, or Minkowski distances are reduced along the feature axis. There is no Python loop over training points. Time complexity is $O(n_q N p)$ and memory complexity is $O(n_q N p)$ for the difference tensor plus $O(n_q N)$ for the distance matrix.

2.3 Sanity check [4 pts]

The implementation is tested against `sklearn.neighbors.KNeighborsClassifier` at seed 42. `pytest` reports **14 passed, 0 failed**; predictions match exactly and `predict_proba` matches within 10^{-9}.

2.4 Probability output [5 pts]

For class c, `predict_proba` implements $\hat{p}(y = c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}\{y^{(i)} = c\}$. Therefore KNN probabilities lie on the grid $\{0, 1/k, 2/k, \ldots, 1\}$. For AUC-ROC this creates ties: many samples can share exactly the same score. My `roc_auc` handles this by sweeping only unique thresholds, computing each tied threshold once, sorting ROC points by FPR, and integrating with the trapezoidal rule.

2.5 Computational benchmark [5 pts]

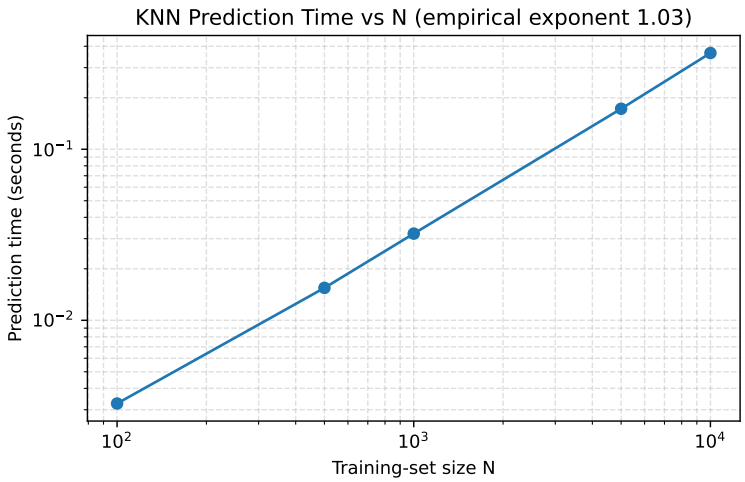


Figure 2: Prediction time vs. training-set size N on a fixed test set of 100 query points.

The empirical log-log scaling exponent is 1.027, close to the theoretical $O(Np)$ behavior for fixed $p = 9$. Small deviations are expected from NumPy overhead and cache effects.

3 Honest Evaluation [30 pts]

3.1 Stratified split [5 pts]

The split sizes are train $(785, 9)$, validation $(261, 9)$, and test $(263, 9)$. Positive-class proportions are 38.2% in train, 38.3% in validation, and 38.0% in test, so the maximum deviation is only 0.3 percentage points, within the 0.5 pp requirement. The function is verified against `train_test_split(stratify=y)`.

3.2 Five metrics from scratch [10 pts]

`src/metrics.py` implements accuracy, precision, recall, F1, and ROC-AUC without sklearn shortcuts. Precision and recall return 0.0 for zero-denominator edge cases, which makes the majority baseline well-defined. The ROC-AUC implementation explicitly handles KNN score ties using unique thresholds. All metrics are verified against sklearn metric functions at fixed seed with numerical tolerance 10^{-9}.

3.3 Hyperparameter sweep [8 pts]

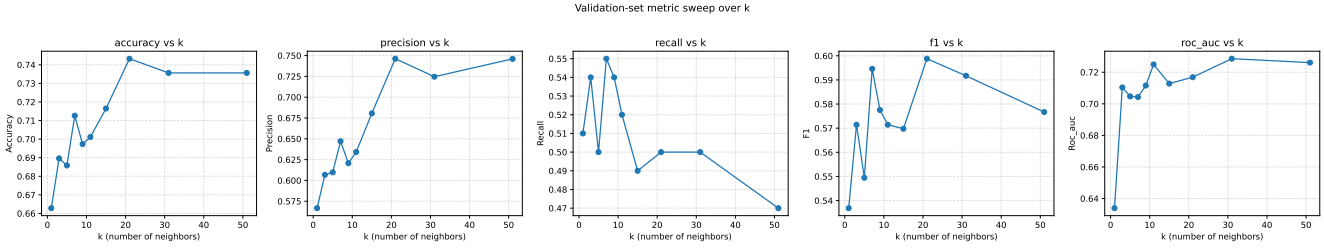


Figure 3: Validation metrics for $k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31, 51\}$.

k	Accuracy	Precision	Recall	F1	AUC
1	0.6628	0.5667	0.5100	0.5368	0.6339
3	0.6897	0.6067	0.5400	0.5714	0.7104
5	0.6858	0.6098	0.5000	0.5495	0.7047
7	0.7126	0.6471	0.5500	0.5946	0.7043
21	0.7433	0.7463	0.5000	**0.5988**	0.7169
31	0.7356	0.7246	0.5000	0.5917	0.7286
51	0.7356	0.7460	0.4700	0.5767	0.7261

Best k by validation F1 is $k = 21$. Recall is highest at smaller k, while larger k improves precision and accuracy; F1 chooses the best balance.

3.4 Baselines [4 pts]

Model	Accuracy	Precision	Recall	F1	AUC-ROC
Majority baseline	0.6169	0.0000	0.0000	0.0000	0.5000
My KNN ($k = 21$)	0.7433	0.7463	0.5000	0.5988	0.7169
sklearn KNN ($k = 21$)	0.7433	0.7463	0.5000	0.5988	0.7172

My KNN strongly outperforms the majority baseline and matches sklearn on predictions. The AUC difference of about 0.0003 is due to implementation-level handling of tied probability thresholds, not a meaningful modelling difference.

3.5 Final test-set evaluation [3 pts]

The test set was used once after fixing $k = 21$ from validation F1.

Split	Accuracy	Precision	Recall	F1	AUC-ROC
Validation	0.7433	0.7463	0.5000	0.5988	0.7169
Test	0.6730	0.5875	0.4700	0.5222	0.7349

The test F1 is lower than validation by 7.7 pp, mainly due to lower precision on the held-out sample. However, test AUC is slightly higher than validation, so the ranking quality is still consistent; the gap is plausible for a validation/test size around 260 passengers and indicates split variance rather than test-set leakage.

4 Cross-Validation, Bias-Variance, and Scaling [20 pts]

4.1 Stratified K-fold CV [5 pts]

`stratified_kfold` returns 5 folds preserving class proportions. It is verified against `StratifiedKFold`; every fold has a positive-class rate close to the global 38.2% rate.

4.2 CV F1 vs. k [5 pts]

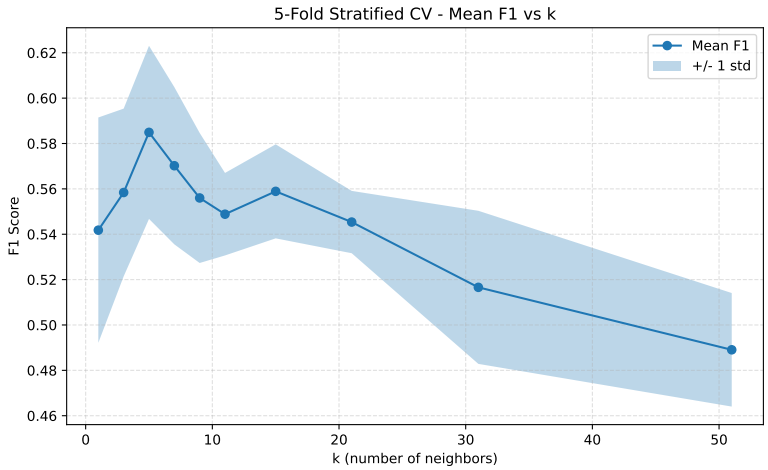


Figure 4: 5-fold stratified CV mean F1 with ±1 standard deviation band.

The best CV choice is $k = 5$ with mean F1 0.5849 and standard deviation 0.0382. The curve falls after the mid-range values: $k = 31$ has mean F1 0.5166 and $k = 51$ has mean F1 0.4891.

4.3 CV-best k vs. single-split-best k [3 pts]

Single validation selects $k = 21$, while 5-fold CV selects $k = 5$. I trust the CV estimate more for general model selection because it averages over five validation folds instead of relying on one random validation partition. The disagreement is also a useful warning: the single split rewarded high precision at $k = 21$, but across folds that choice is less stable than $k = 5$.

4.4 Bias-variance discussion [4 pts]

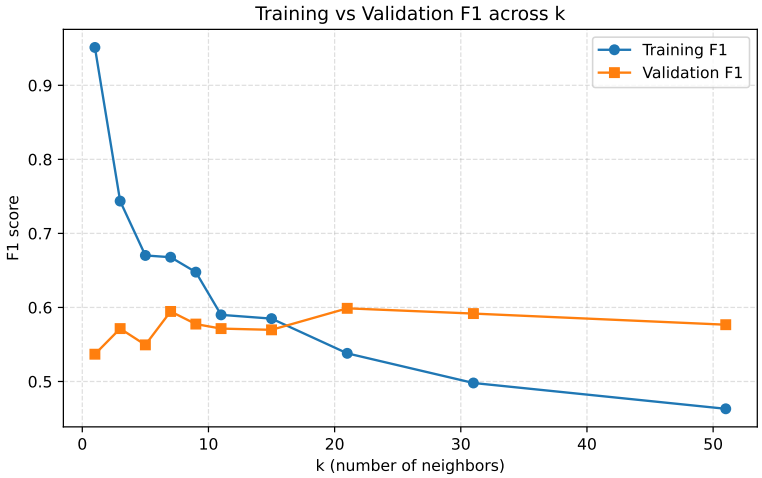


Figure 5: Training vs validation F1 across k.

At $k = 1$, training F1 is 0.9511 while validation F1 is only 0.5368, a large generalization gap that indicates overfitting. Increasing k smooths the decision boundary and lowers variance: by $k = 7$ validation F1 improves to 0.5946 while training F1 falls to 0.6678. At very large k, the model becomes too smooth; CV F1 drops from 0.5849 at $k = 5$ to 0.4891 at $k = 51$, showing underfitting.

4.5 Scaling experiment [3 pts]

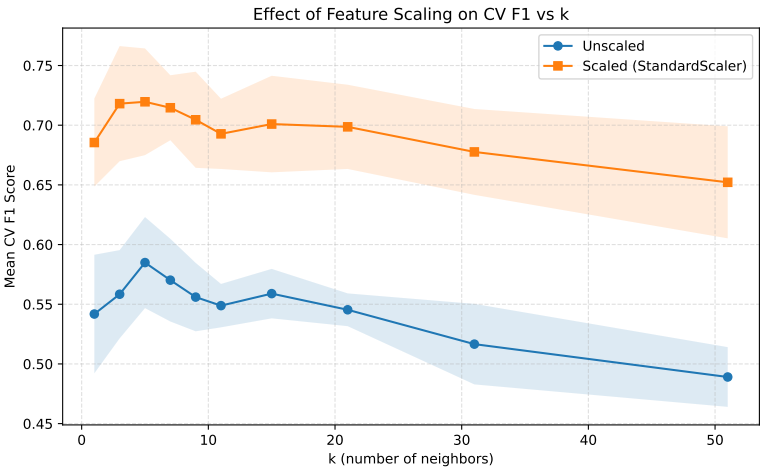


Figure 6: Effect of StandardScaler on CV F1; scaler is fit only on the training fold.

Without scaling, best mean CV F1 is 0.5849 at $k = 5$. With StandardScaler fit only on each training fold, best mean CV F1 becomes 0.7196 at $k = 5$, a lift of +13.47 percentage points. KNN is scale-sensitive because Euclidean distance uses raw numeric ranges: fare ranges up to about 512, while binary variables such as sex and embarked range only from 0 to 1. Without scaling, fare dominates distance; after scaling, sex, pclass, age, and embarked can contribute meaningfully.

Bonus note

Weighted KNN is implemented with `weights="distance"` using weights $1/(d + 10^{-8})$, and the equidistant case is tested so it reduces to uniform voting. `ApproxKNN` is also provided as a BallTree wrapper with the same public API, but the core report grade does not rely on the optional large synthetic benchmark.

Reproducibility

All random operations are seeded with 42 unless otherwise stated. Running `python src/experiments.py` from the project root regenerates the figures and prints the same metric tables; running `pytest -q` gives 14 passed tests.